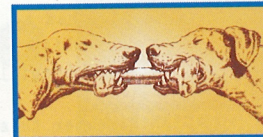


PROGRAMMERS' CHALLENGE

By Mike Scanlin, MacTech Magazine Regular Contributing Author



ROTATED BITS

This month's challenge is to write a routine that quickly rotates a 1-bit deep bitMap 90 degrees clockwise. You are given pointers to a source bitMap and destination bitMap. Space for the destination pixels has already been allocated (the baseAddr field is valid) and the rowBytes and bounds fields have been set up for you. All you have to do is rotate the bits.

The function prototype you write is:

```
void
RotateBitMapClockwise(srcBitMapPtr,
dstBitMapPtr)
BitMap *srcBitMapPtr;
BitMap *dstBitMapPtr;
```

To check for correctness, you might want to write a test program that rotates the same bitMap four times and then diffs the result with the original (they should be identical). Have fun.

FEBRUARY WINNER

The winner of the "Insane Anglo Warlord" challenge is **Jeremy Vineyard** (Lawrence, KS) whose solution was the only one of the two I received which gave valid results (the other one gave intermediate strings that had a character not in the original strings). Remember kids: you have to play the game to have

a chance at winning.

Here's the output of Jeremy's Descramble routine when the input strings are "INSANE ANGLO WARLORD" and "RONALD WILSON REAGAN" when the number of intermediate steps is set to 10:

```
INSANE ANGLO WARLORD
OIDSLNE N NGWRALORAA
ORDLINE AWNSNLORAAAG
R AODSILEWANN ORALGN
R WADOSGLNIAA RLOENN
R IWODNSALRAAL OEGNN
ROD INLASL RAWOEAGNN
RONDIL A SLORWAEAGNN
RONLDLI S AORWAEAGNN
ROND L LWASIOR AEAGNN
RONWLD AILSOR AEAGNN
RONALD WILSON REAGAN
```

And here is Jeremy's code to generate those strings:

```
/******
 * Descramble.c
 * A procedure that returns a smooth
 * transition from one string to
 * another with the same characters,
 * but in a different order.
 * *****/

#include <stdlib.h>
#include <string.h>

void Descramble(
```

```
Str255 startString,
Str255 endString,
unsigned short numSteps,
Str255 *stepStringPtrs[20] )
{
    short i, j, k;
    short strLength = startString[0];
    div_t divResult;
    short destination, memSwitch;
    Boolean switchPossible;

    short distances[256];
    short destMap[256];
    Boolean destSwitchMap[256];
    Boolean checkList[256];
```

```
/* Clear the destination switching
 * map for use. Only clear it up to
 * strLength to save time. */
for (i = 1; i <= strLength; i++)
    destSwitchMap[i] = false;

/* This loop searches for a duplicate
 * char 'j' for the char 'i'. Once it
 * finds a duplicate, it checks to
 * see whether it is already being
 * used as another char's
 * destination. If not, it shows that
 * char 'j' is char 'i's destination,
 * sets the destMap to show that char
 * 'j' is being used, and
 * calculates how far the char 'i'
 * needs to travel on
 * each step to get to its
 * destination. */
for (i = 1; i <= strLength; i++)
    for (j = 1; j <= strLength; j++)
        if ( startString[i] ==
            endString[j] &&
            ! destSwitchMap[j] )
        {
            /* If destSwitchMap[j] is set,
```

Here's how it works: Each month there will be a different programming challenge presented here. First, you must write some code that solves the challenge. Second, you must optimize your code (a lot). Then, submit your solution to MacTech Magazine (formerly MacTutor). A winner will be chosen based on code correctness, speed, size and elegance (in that order of importance) as well as the postmark of the answer. In the event of multiple equally desirable solutions, one winner will be chosen at random (with honorable mention, but no prize, given to the runners up). The prize for the best solution each month is \$50 and a limited edition "The Winner! MacTech Magazine Programming Challenge" T-shirt (not to be found in stores).

In order to make fair comparisons between solutions, all solutions must be in ANSI compatible C. All entries will be tested with the FPU and 68020 flags turned off in THINK C. When timing routines, the latest version of THINK C will be used (with ANSI Settings plus "Honor 'register' first" and "Use Global Optimizer" turned on) so beware if you optimize for a different C compiler.

THE RULES

The solution and winners for this month's Programmers' Challenge will be published in the issue two months later. All submissions must be **received** by the 10th day of the month printed on the front of this issue.

All solutions should be marked "Attn: Programmers' Challenge Solution" and sent to Xplain Corporation (the publishers of MacTech Magazine) via "snail mail" or preferably, e-mail - AppleLink: MT.PROGCHAL, Internet: progchallenge@xplain.com, and CompuServe: 71552,174. If you send via snail mail, please include a disk with the solution and all related files (including contact information). See page 2 for information on "How to Contact Xplain Corporation."

MacTech Magazine reserves the right to publish any solution entered in the Programming Challenge of the Month and all entries are the property of MacTech Magazine upon submission. The submission falls under all the same conventions of an article submission.

PROGRAMMERS' CHALLENGE

By Mike Scanlin, MacTech Magazine Regular Contributing Author



WHERE IN THE WORLD?

This month's Challenge is a real world problem previously solved without reals by real MacTech reader Allen Stenger (Gardena, CA): You are given a large array of unique city names in alphabetical order ending with a sentinel of 'ZZZ' and a single city name to find in this array. You are to return the index of the desired city, or if it doesn't match exactly, a list of the five closest matches. If there are more than five closest ones, return any five of them. The closeness-of-match algorithm, for purposes of this challenge, is "differing in the fewest positions from the desired name." Take the longest of the two strings you're comparing and each time a position doesn't match (or is beyond the end of the shorter string) add 1 to your difference count; the smallest difference count is considered closest.

The input city list is an array of Str255s (pascal strings) in one big block where there will be, say, 200 to 1000 unique city names (in alphabetical order, all uppercase, with the 'ZZZ' sentinel as the last element). The city you are to find is the uppercase string pointed to by cityToFindNamePtr. The

prototype of the function you write is:

```
Boolean FindCity(cityNames,
cityToFindNamePtr, closestMatches)
Str255          cityNames[];
Str255          *cityToFindNamePtr;
unsigned short  closestMatches[];
```

If there is an exact match, return TRUE and set closestMatches[0] equal to the zero-based index of the match. If there is not an exact match, return FALSE and set closestMatches[0..4] to the five closest matches (in order of closeness, or, if they are equally close, in any order).

Allen says that this problem came up while working on the American Airlines reservation system, SABRE, a long time ago. He says that his tests then showed that city names were misspelled about 1/3rd of the time due to either poor spelling or typing by the reservation agents (no offense intended). You can use that information in your solution if you like; I will be simulating it with test data (exact matches will occur only 33% of the time I call your routine from my test bench).

TWO MONTHS AGO WINNER

Congratulations to Steve Israelson (Vancouver, B.C.) for winning the "Rotated

Bits" challenge. Right on his heels is Andy Scheck (location unknown) with an entry that was half as many bytes but which was just a little too slow in some cases to win. Jeff Mallett (Hickory, NC) deserves praise for his extremely small solution which was also near the top in terms of speed. Because it is so small, it is listed here as well, after Steve's winning solution.

There were 16 entries to this challenge. Two of them were disqualified immediately because they were in assembly (nice try) and six others were disqualified for giving incorrect results (try testing with a 9x17 bitmap). Many people e-mailed me and asked if they could assume that rowBytes was divisible by four. The answer is: no. However, you could always case out on that case early on in your routine and then call one of two rotate functions based on if (rowBytes % 4 == 0) or not. Just so long as your solution has one entry point you can do whatever you like inside.

Here's a summary of the eight entries that gave correct results. The bytes column is the code size, test 1 is the ticks to rotate 4000 small bitmaps (20x30 and smaller) and test 2 is the ticks

THE RULES

Here's how it works: Each month there will be a different programming challenge presented here. First, you must write some code that solves the challenge. Second, you must optimize your code (a lot). Then, submit your solution to MacTech Magazine (formerly MacTutor). A winner will be chosen based on code correctness, speed, size and elegance (in that order of importance) as well as the postmark of the answer. In the event of multiple equally desirable solutions, one winner will be chosen at random (with honorable mention, but no prize, given to the runners up). The prize for the best solution each month is \$50 and a limited edition "The Winner! MacTech Magazine Programming Challenge" T-shirt (not to be found in stores).

In order to make fair comparisons between solutions, all solutions must be in ANSI compatible C (i.e., don't use Think's Object extensions). **Only pure C code can be used. Any entries with any assembly in them will be disqualified.** However, you may call any routine in the Macintosh toolbox you want (i.e., it doesn't matter if you use NewPtr instead of malloc). All entries will be tested with the FPU and 68020 flags turned off in THINK C. When timing routines, the latest version of THINK C will be used (with ANSI Settings plus "Honor register first" and "Use Global Optimizer" turned on) so

beware if you optimize for a different C compiler. **All code should be limited to 60 characters wide. This will aid us in dealing with e-mail gateways and page layout.**

The solution and winners for this month's Programmers' Challenge will be published in the issue two months later. All submissions must be **received** by the 10th day of the month printed on the front of this issue.

All solutions should be marked "Attn: Programmers' Challenge Solution" and sent to Xplain Corporation (the publishers of MacTech Magazine) via "snail mail" or preferably, e-mail - AppleLink: MT.PROGCHAL, Internet: progchallenge@xplain.com, and CompuServe: 71552,174. If you send via snail mail, please include a disk with the solution and all related files (including contact information). See page 2 for information on "How to Contact Xplain Corporation."

MacTech Magazine reserves the right to publish any solution entered in the Programming Challenge of the Month and all entries are the property of MacTech Magazine upon submission. The submission falls under all the same conventions of an article submission.

to rotate 400 medium size bitmaps (300x400 pixels, approx). In all cases, the bitmaps were filled with the same random data before rotating:

Name	bytes	test 1	test 2
Steve Israelson	740	72	638
Andy Scheck	386	75	778
Jeff Mallett	194	91	936
Patrick Breen	668	125	902
Stepan Riha	656	122	1186
Dave Darrah	362	138	1453
Jan Bruyndonckx	490	205	1785
Dominic Mazzoni	244	630	6687

Steve's winning solution is reasonably commented so I won't go into detail here describing how it works. I will, however, offer some peephole optimizations to make it even better.

The first thing that the optimized eye notices about his code is that he multiplies, divides and mods by 8. It's a dangerous assumption to make that the compiler will be smart and substitute shifts and ands where it can. I always prefer to make them explicit. Thus, I would change these lines:

```
srcRowX8 = srcRowBytes * 8;
srcData = const1 + srcX / 8;
if (remainder = (srcWidth % 8))
```

to these:

```
srcRowX8 = srcRowBytes << 3;
srcData = const1 + (srcX >> 3);
if (remainder = (srcWidth & (8 - 1)))
```

In addition to those, there is one point in his code where he multiplies by 7. He'd be better off by multiplying by 8 (using a shift) and then subtracting one. This:

```
src += srcOffset * 7;
```

is faster as:

```
src += srcOffset << 3;
src -= srcOffset;
```

After I made these changes and then ran test 2, the time went from 638 ticks to 542 ticks (it reduced the code size by 18 bytes, too).

Here's Steve's winning solution, followed by Jeff's tiny solution:

```
/*-----
Steve Israelson
Motion Works Corp.
-----*/
void RotateBitMapClockwise(Bitmap *, Bitmap *);
void rotateBlock(unsigned long *, unsigned char *,
long, unsigned char *, long);
void rotateLimitedBlock(unsigned long *,
```

```
unsigned char *, long, unsigned char *, long,
short);
```

```
/*-----
Rotate the bitmap from src to dst by +90 degrees. Make
a table for use in the rotate block routine. Step through
the src in 8x8 blocks, starting from the bottom left. Step
through dst in 8x8 blocks, starting from the top left.
Handle the right edge of the bitmap specially to prevent bits
from being rotated into memory outside of the destination.
-----*/
```

```
void RotateBitMapClockwise(Bitmap *src, Bitmap *dst)
{
```

```
unsigned long table[16] =
{0x00000000, 0x00000001, 0x00000100, 0x00000101,
0x00010000, 0x00010001, 0x00010100, 0x00010101,
0x01000000, 0x01000001, 0x01000100, 0x01000101,
0x01010000, 0x01010001, 0x01010100, 0x01010101};
```

```
/* x,y co-ord of source block */
register short srcX, srcY;
```

```
/* row bytes, dereferenced for speed */
```

```
short srcRowBytes, dstRowBytes;
```

```
/* src rowbytes * 8, for speed */
```

```
short srcRowX8;
```

```
/* width and height of src */
```

```
short srcWidth, srcHeight;
```

```
/* pointer to src and dst 8x8 blocks */
```

```
char *srcData, *dstData;
```

```
/* pointer to bitmaps de-referenced for speed */
```

```
char *srcAddr, *dstAddr;
```

```
/* last few pixels to do specially */
```

```
short remainder;
```

```
/* a constant, pre-calculated for speed */
```

```
char *const1;
```

```
/* de-reference some constants */
```

```
srcRowBytes = src->rowBytes;
```

```
dstRowBytes = dst->rowBytes;
```

```
srcAddr = src->baseAddr;
```

```
dstAddr = dst->baseAddr;
```

```
/* calculate some values */
```

```
srcWidth = src->bounds.right - src->bounds.left;
```

```
srcHeight = src->bounds.bottom - src->bounds.top;
```

```
srcRowX8 = srcRowBytes * 8;
```

```
const1 = srcAddr + (srcHeight - 8) * srcRowBytes;
```

```
/* loop through the source doing vertical
```

```
* slices starting at the left */
```

```
for (srcX = 0; srcX < srcWidth - 7; srcX += 8) {
```

```
/* bottom edge */
```

```
srcData = const1 + srcX / 8;
```

```
/* left edge */
```

```
dstData = dstAddr + srcX * dstRowBytes;
```

```
for (srcY = srcHeight - 1; srcY >= 0;
```

```
srcY -= 8) {
```

```
rotateBlock(table, (unsigned char *)srcData,
```

```
srcRowBytes, (unsigned char *)dstData,
```

```
dstRowBytes);
```

```
srcData += srcRowX8; /* next vert block */
```

```
++dstData; /* next horiz block */
```

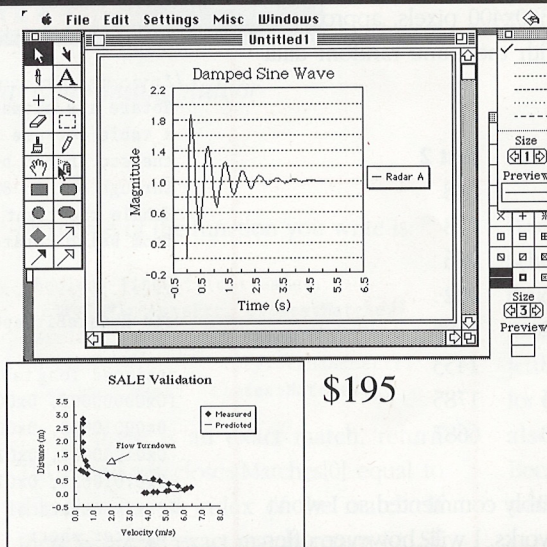
```
}
```

```
}
```

Plot while programming

SuperPlot™

Easy-to-use plotting library
Think C and Pascal
MPW C and Pascal
Fortran



- Simple subroutine call
- Presentation quality plots
- Complete Macintosh interface
- 14 plot types
- Overlay up to 10 plots
- Edit, print, and export plots
- Interactive Data Editing
- Templates

Call 1-800-827-4351

SuperSoft



498 E. Robin Rd.
Orem, UT 84057
(801) 225-4356
Fax: (801) 226-6276

\$195

```
/* handle the last partial row by calling
 * rotateLimitedBlock() */
if (remainder = (srcWidth % 8)) {
    /* bottom edge */
    srcData = const1 + srcX / 8;
    /* left edge */
    dstData = dstAddr + srcX * dstRowBytes;
    for (srcY = srcHeight - 1; srcY >= 0;
        srcY -= 8) {
        rotateLimitedBlock(table,
            (unsigned char *)srcData, srcRowBytes,
            (unsigned char *)dstData, dstRowBytes,
            remainder);
        srcData -= srcRowX8;
        ++dstData;
    }
}
```

/*-----
Rotate an 8x8 block of pixels by treating the destination block as 2 unsigned longs (64 bits!) and using a lookup table to get a mask for the value of each row in the src block. The table essentially stretches out the source byte to 8 times its size and then ORs it into the destination 64bits. Since the bits are in the low bit of each byte, the destination has to be shifted one position to the left before each OR operation. The offset values are the difference to rowBytes. If we had a 64bit data type, this would be easier.
-----*/

```
void rotateBlock(unsigned long *table,
    unsigned char *src, long srcOffset,
    unsigned char *dst, long dstOffset)
{
    register unsigned long resultLO, resultHi;
    short x;

    resultLO = resultHi = 0;
    src += srcOffset * 7;
    /* compute the rotated result */
```

```
for (x = 0; x < 8; ++x) {
    resultLO = resultLO << 1;
    resultHi = resultHi << 1;
    resultLO |= table[(src) >> 4];
    resultHi |= table[(src) & 0x0F];
    src -= srcOffset;
}
```

```
/* store the rotated result */
*dst = (resultLO & 0xFF000000) >> 24;
dst += dstOffset;
*dst = (resultLO & 0x00FF0000) >> 16;
dst += dstOffset;
*dst = (resultLO & 0x0000FF00) >> 8;
dst += dstOffset;
*dst = resultLO & 0x000000FF;
dst += dstOffset;
*dst = (resultHi & 0xFF000000) >> 24;
dst += dstOffset;
*dst = (resultHi & 0x00FF0000) >> 16;
dst += dstOffset;
*dst = (resultHi & 0x0000FF00) >> 8;
dst += dstOffset;
*dst = resultHi & 0x000000FF;
```

/*-----
Same as other rotateBlock, except will do a partial block. Useful to prevent the overwriting of memory outside the bitmap!
-----*/

```
void rotateLimitedBlock(unsigned long *table,
    unsigned char *src, long srcOffset,
    unsigned char *dst, long dstOffset, short lines)
{
    register unsigned long resultLO, resultHi;
    short x;

    resultLO = resultHi = 0;
    src += srcOffset * 7;
    for (x = 0; x < 8; ++x) {
        resultLO = resultLO << 1;
```

StoneTable™ a drop-in replacement library for the List Manager

resize
titles
variable height rows
variable width columns
move/copy row/column
default titles of letters or numbers
sort rows/columns
"LDEF-like" custom
drawing support

	Name	Parents
		Address
Father	Joe Smith	2138 NE Halsed #3 Portland, OR 97232
Mother	Sue Smith	436 Miller Creek San Rafael, CA 92013

edit cell/title in place
different font per cell
appl. defined cell validation
hide rows/columns
32K text per cell
> 32K non-text per cell
scroll cell text

Libraries available for:
Think C \$150
MPW C \$150
Think Pascal \$150

StoneTable Publishing
P.O. Box 12665
Portland, OR 97212
(503) 287-3424
Internet: 70303.2546@compuserve.com
CIS: 70303,2546

All the functions of the List Manager
plus support for those listed above and more.
Simple conversion from the List Manager.

International shipping (US Airmail) \$10
Payment only by check in US\$
No royalty fees for applications
TCL/MacApp not required

```

resultHi = resultHi << 1;
resultLO |= table[*src] >> 4];
resultHi |= table[*src] & 0x0F];
src -= srcOffset;
)

*dst = (resultLO & 0xFF000000) >> 24;
dst += dstOffset; if (lines == 1) return;
*dst = (resultLO & 0x00FF0000) >> 16;
dst += dstOffset; if (lines == 2) return;
*dst = (resultLO & 0x0000FF00) >> 8;
dst += dstOffset; if (lines == 3) return;
*dst = resultLO & 0x000000FF;
dst += dstOffset; if (lines == 4) return;
*dst = (resultHi & 0xFF000000) >> 24;
dst += dstOffset; if (lines == 5) return;
*dst = (resultHi & 0x00FF0000) >> 16;
dst += dstOffset; if (lines == 6) return;
*dst = (resultHi & 0x0000FF00) >> 8;
dst += dstOffset; if (lines == 7) return;
*dst = resultHi & 0x000000FF;
)

//*****
// RotateBitMapClockwise
// By Jeff Mallett
//
// Rotates the significant bits of the srcBitMapPtr 90°
// clockwise and stores the result in dstBitMapPtr.
//*****

#define kHighShortBit    0x8000
#define kBitsPerShort    16

// Creates a short from bits in different rows of
// the source bitmap. Then copies this short into
// the destination bitmap.
#define COPY_TWO_BYTES(stopValue) \
data = 0, bit = kHighShortBit; \
do { \
    if (*srcPos & srcBit) data |= bit; \

```

```

srcPos += srcRowShorts; \
} while ( (bit >= 1) != stopValue ); \
*(dstPos++) = data

void RotateBitMapClockwise(BitMap *srcBitMapPtr,
BitMap *dstBitMapPtr)
{
    register unsigned short data, *srcPos, bit;
    register int j;
    register unsigned short srcBit, *dstPos, stopBit;
    register int i;
    unsigned short *baseSrcPtr;
    // shorts per row of source
    const int srcRowShorts =
        srcBitMapPtr->rowBytes >> 1;
    const int numSrcRows =
        srcBitMapPtr->bounds.bottom - srcBitMapPtr->bounds.top;

    dstPos = (unsigned short *)dstBitMapPtr->baseAddr;
    srcPos = baseSrcPtr =
        (unsigned short *)srcBitMapPtr->baseAddr;
    srcBit = kHighShortBit;

    for (i = srcBitMapPtr->bounds.right -
        srcBitMapPtr->bounds.left; i; --i) {
        // Copy one column of source to a row of destination
        for (j = numSrcRows / kBitsPerShort; j; --j) {
            COPY_TWO_BYTES(0);
        }
        if (j = numSrcRows % kBitsPerShort) {
            stopBit = kHighShortBit >> j;
            COPY_TWO_BYTES(stopBit); // Final 2 bytes
        }
        // Prepare to copy next column of source
        if ( !(srcBit >= 1) ) {
            srcBit = kHighShortBit;
            ++baseSrcPtr;
        }
        srcPos = baseSrcPtr;
    }
}

```

